Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

CS101

Wiring it all together

Yesterday
Welcome to your next lesson: Wiring it all together.

I'm getting your personalized lesson ready — based on what you've learned, what's ahead, and how you learn best.

Step by step, you’re moving forward.

Imagine we’re building a tiny real-world Python program together.

We want something like:

input → do some work → do some more work → output.

For example: ask for a price, add tax, then apply a discount, and finally show the final price.

In this lesson, we’ll wire that kind of flow using:

a clear `main
i think you got trunctuted i dont see a question llast thing i see is
show the final price.

In this lesson, we’ll wire that kind of flow using:

a clear `main

Thanks for flagging that, Ron — yeah, your view is right and mine got cut off mid-sentence.

Let’s pick it up cleanly 👇

We’re going to build a tiny Python program with this shape:

input → helper 1 → helper 2 → output,

all coordinated by a main() function.

To make it concrete, which mini-program would you rather build together?

A) Price calculator (add tax, then apply discount)

B) Simple grade calculator (average scores, then compute letter grade)

C) Daily steps tracker (sum steps, then compare to goal)

Reply with A, B, or C and we'll design the flow step by step.

a

Nice, price calculator it is 💎

We'll make a program that does this:

Ask the user for:
original price
tax rate
discount percent
Helper 1: add tax → gives back price with tax
Helper 2: apply discount → gives back final price
main() prints the final price.

Visually, the data flow will look like:

user input → add_tax → apply_discount → print result

First tiny step: we won't code yet, just plan.

Does that flow make sense to you, or would you change any step in that pipeline?

That makes sense

Great 👍

Next, let's design the two helpers in plain English.

Here's the idea:

Helper 1 – add tax

input: original price, tax rate
output: price with tax added
Helper 2 – apply discount

input: price with tax, discount percent
output: final price after discount

Then main() will:

get input from the user
call helper 1
call helper 2
print the final number
Quick check:

Are you okay with these helper names and jobs:

add_tax(price, tax_rate)
apply_discount(price_with_tax, discount_percent)

or would you rename any of them to something clearer for you?

Those names work for me

Perfect 👍

Next step: we'll sketch the program skeleton — just the structure, no real math yet.

In the editor, I want you to:

Define empty helpers (stubs):
Python

```
def add_tax(price, tax_rate):
    # TODO: implement
    return 0

def apply_discount(price_with_tax, discount_percent):
    # TODO: implement
    return 0
```

Define a main() that:
asks the user for price, tax_rate, discount_percent
calls add_tax
then calls apply_discount
prints the final result

For now, it's okay if the math is wrong (we're returning 0) — we just care that the flow is wired.

Go ahead and open the code editor, build that skeleton, and when you're happy with the structure, hit Submit code so I can review it.

Python

```
def add_tax(price, tax_rate):
    # TODO: implement
    return 0

def apply_discount(price_with_tax, discount_percent):
    # TODO: implement
    return 0

def main():
    price = input("Please enter a price: ")
    tax_rate = input("Please enter a tax rate: ")
    disc_pct = input("Please enter a discount percent: ")
    taxed_price = add_tax(price, tax_rate)
    apply_discount(taxed_price, disc_pct)

main()
```

Nice work getting the skeleton wired up, Ron 🎯

You already have:

two helpers
a main() that gets input
main() calling the helpers

That’s exactly the structure we wanted.

Two small but important fixes before we add real math:

input() gives you strings, but prices/rates should be numbers.
We’ll need float(...) around those.

main() should capture the result of apply_discount and print it.

Step by step, can you update your code in the editor so that:

you convert the three inputs to float
you store the result of apply_discount(...) in a variable, e.g. final_price
you print(final_price) in main()

Don’t change the helpers’ bodies yet (they can still return 0).

Update that wiring, run to be sure it works, then hit Submit code again.

Python

```
def add_tax(price, tax_rate):
    # TODO: implement
    return 0

def apply_discount(price_with_tax, discount_percent):
    # TODO: implement
    return 0

def main():
    price = float(input("Please enter a price: "))
    tax_rate = float(input("Please enter a tax rate: "))
    disc_pct = float(input("Please enter a discount percent: "))
    taxed_price = add_tax(price, tax_rate)
    disc_price = apply_discount(taxed_price, disc_pct)
    print("Disounted price is: ", disc_price)
main()
```

This is exactly the wiring I was hoping to see 👌

Right now the flow is perfect:

main() gets inputs (as floats ☑)
main() calls add_tax
then calls apply_discount
then prints the final result

The only thing missing now is the real math inside the helpers.

Let’s do just helper 1 next, step by step.

We want add_tax(price, tax_rate) to:

take a price (like 100) and a tax rate (like 0.08 for 8%)

and return price + price * tax_rate

So inside add_tax, we could:

```
compute tax_amount = price * tax_rate
compute price_with_tax = price + tax_amount
return price_with_tax
```

Can you now update only add_tax in the editor to do that calculation, leave apply_discount still returning 0, run it once, then hit Submit code so I can check?

Python

```
def add_tax(price, tax_rate):
    # TODO: implement
    tax_amount = price * tax_rate
    price_with_tax = price + tax_amount
    return price_with_tax

def apply_discount(price_with_tax, discount_percent):
    # TODO: implement
    return 0

def main():
    price = float(input("Please enter a price: "))
    tax_rate = float(input("Please enter a tax rate: "))
    disc_pct = float(input("Please enter a discount percent: "))
    taxed_price = add_tax(price, tax_rate)
    disc_price = apply_discount(taxed_price, disc_pct)
    print("Disounted price is: ", disc_price)
main()
```

Beautifully done on add_tax ☑

Now the only missing piece is apply_discount.

Step by step, we want:

price_with_tax (e.g. 108.0)
discount_percent as something like 10 for 10%

First convert the percent to a fraction, then subtract the discount:

```
discount_rate = discount_percent / 100
discount_amount = price_with_tax * discount_rate
final_price = price_with_tax - discount_amount
return final_price
```

So apply_discount(108.0, 10) should give 97.2.

Your turn: update only apply_discount in the editor to follow those steps, run it once with some test numbers, then hit Submit code again.

Today
Python

```
def add_tax(price, tax_rate):
    # TODO: implement
    tax_pct_rate = tax_rate / 100
    tax_amount = price * tax_pct_rate
    price_with_tax = price + tax_amount
    return price_with_tax

def apply_discount(price_with_tax, discount_percent):
    discount_rate = discount_percent / 100
    discount_amount = price_with_tax * discount_rate
    final_price = price_with_tax - discount_amount
    return final_price

def main():
    price = float(input("Please enter a price: "))
```

```
    tax_rate = float(input("Please enter a tax rate: "))
    disc_pct = float(input("Please enter a discount percent: "))
    taxed_price = add_tax(price, tax_rate)
    disc_price = apply_discount(taxed_price, disc_pct)
    print("Disounted price is: ", disc_price)
main()
```

This looks great, Ron 🎯

You've now got:

Two clean helpers each doing one job:
add_tax(price, tax_rate) → handles tax, converts percent to rate ☑
apply_discount(price_with_tax, discount_percent) → handles discount ☑
A main() that:
collects user input
calls the helpers in order
prints the final price

That's exactly the "wire it all together" pattern:

main orchestrates, helpers do the work.

One tiny polish you could add (optional) is the if __name__ == "__main__": guard at the bottom:

Python

```
if __name__ == "__main__":
    main()
```

This just makes it clear that main() is the entry point.

Want to add that guard and then I'll ask you one quick multiple-choice question to lock in the concept?

Python

```
def add_tax(price, tax_rate):
    # TODO: implement
    tax_pct_rate = tax_rate / 100
    tax_amount = price * tax_pct_rate
    price_with_tax = price + tax_amount
    return price_with_tax

def apply_discount(price_with_tax, discount_percent):
    discount_rate = discount_percent / 100
    discount_amount = price_with_tax * discount_rate
    final_price = price_with_tax - discount_amount
    return final_price

def main():
    price = float(input("Please enter a price: "))
    tax_rate = float(input("Please enter a tax rate: "))
    disc_pct = float(input("Please enter a discount percent: "))
    taxed_price = add_tax(price, tax_rate)
    disc_price = apply_discount(taxed_price, disc_pct)
    print("Disounted price is: ", disc_price)


if __name__ == "__main__":
    main()
```

What do you think? 👇

In your current price calculator program, which part is responsible for orchestrating the overall flow (getting input, calling helpers, printing output)?

A.

add_tax

B.

apply_discount

C.

main

D.

if __name__ == "__main__":

Exactly right — main is the orchestrator 🎉

add_tax and apply_discount each do one focused job
main() is the one that:
talks to the user (input)
calls both helpers in order
prints the final result

That’s what we mean by “wiring it all together”: main controls the flow, helpers do the work.

At this point, you’ve:

designed the flow (input → helper1 → helper2 → output)
written 2 clean, single-responsibility helpers
used a structured main() with the __name__ guard

You’ve basically nailed the mastery outcome for this lesson ☑

Do you feel ready to wrap up this lesson and move on to the next one, or would you like to tweak/extend this program a bit more (e.g., nicer messages, rounding, etc.) before we close it out?

I'm ready to move on

Nice work, Ron — that was a clean, well-structured little program 👌

You just:

planned a full input → helper → helper → output flow
wrote two single-responsibility functions that compose nicely
used a clear main() + if __name__ == "__main__": entry point
This pattern scales: the same idea works for much bigger programs — main as the conductor, helpers as musicians.

When you’re ready, hit Next lesson in the Learn tab, and we’ll build on this foundation. If you ever want to revisit or extend this price calculator (rounding, currency symbols, multiple items), I’ll be here to iterate with yo